SYSTEMS MANAGEMENT



Runtime SBOM (Software Bill of Materials)

From build-time software composition to operational vulnerability management

Harsh Bhuwania

Senior Firmware Engineer, Microsoft

Prashant Dewan

Principal Software Engineer, Microsoft

OCP PLATINUM® Member

Vulnerability volume keeps climbing

48,244

CVEs published in 2025, about 20% more than in 2024. [1]

Hours

Synack reports that AI-enabled attackers now exploit newly disclosed vulnerabilities within hours. [2]

For an operator, the first question after each disclosure is whether it touches anything in the fleet.

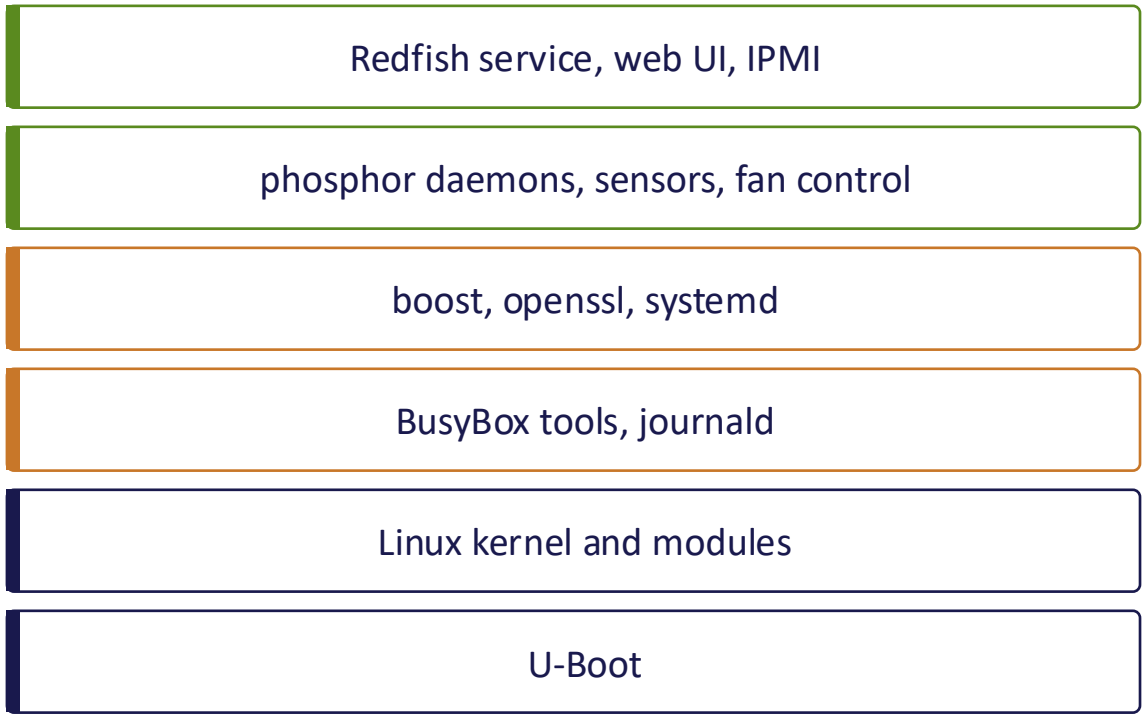
[1] CVE.org count of published CVEs for 2025, as cited in Synack, 2026 State of Vulnerabilities Report (May 14, 2026) [2] Synack, 2026 State of Vulnerabilities Report (May 14, 2026)

What is an SBOM?

A Software Bill of Materials is a machine-readable ingredients list for software.

- An ingredients list for software
- Components and versions
- Dependency relationships
- Suppliers, licenses, hashes
- Available in SPDX or CycloneDX format

What that looks like in a BMC image



An SBOM tells you what is inside. Knowing if you are exposed means matching it against CVE feeds and advisories. **Today there is no standardized, interoperable method to associate deployed firmware with the correct SBOM across vendors and management ecosystems.**

What is inside one compute node

Illustrative example: a Yocto-based BMC firmware image (rounded figures)

~500
components

~300
third-party

~200
upstream projects

25,000+
hashed files



Package-level identifiers exist, but operational systems still need a way to associate deployed assets with the correct SBOM.

This is one controller. The same machine also carries BIOS, CPLD, NICs, drives, and power supplies, each with its own supplier.

Built is not the same as deployed

BUILD-TIME VIEW

Firmware build

A released software or firmware artifact

SBOM generated

The composition record is stored with the release

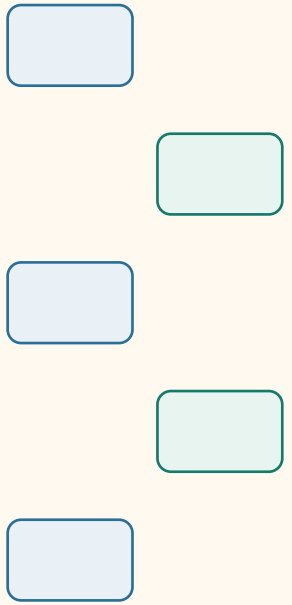
Artifact released

Build-time association is clear

?

OPERATIONAL REALITY

- Multiple releases
- Configuration variants
- Active, staged, and recovery
- Long-lived assets
- New intelligence



Why not just use the version?

It gets you close, but breaks on configuration variants, on rebuilds that do not change the version number, and on identifier formats that differ by supplier.

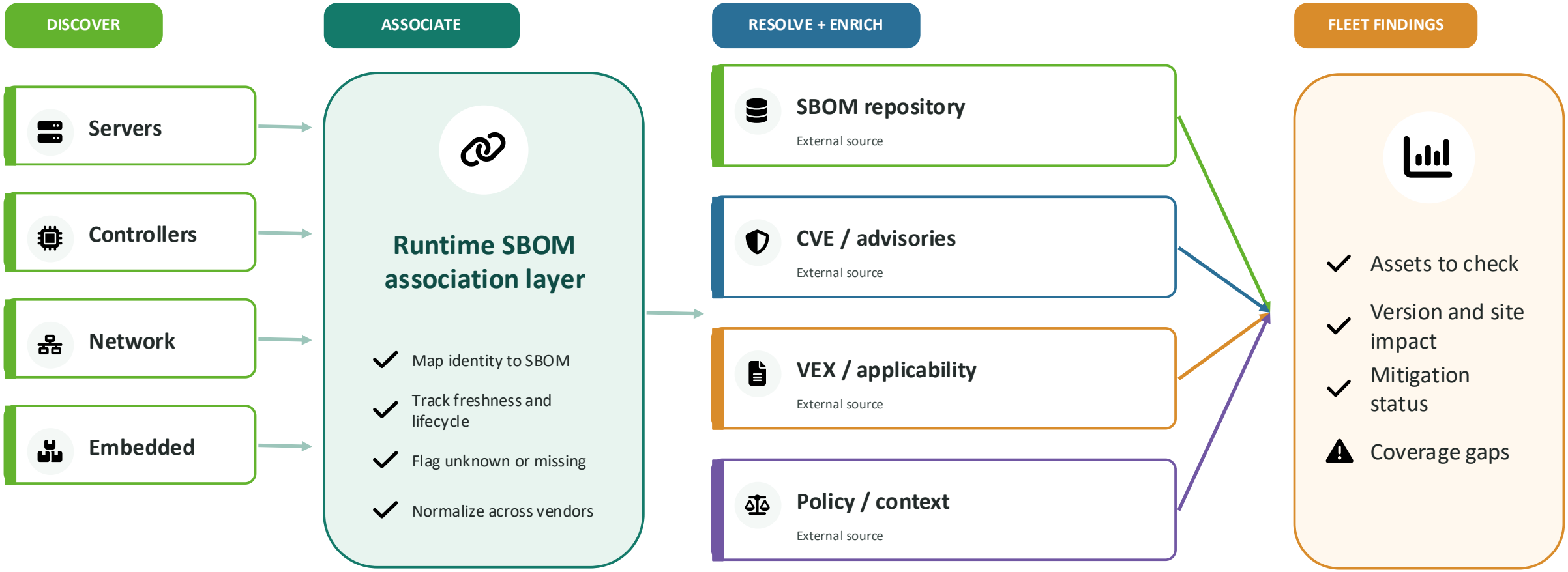
Missing association: Which SBOM applies to the software or firmware deployed on each managed asset?

What Runtime SBOM visibility means

Runtime SBOM visibility is knowing which SBOM belongs to the firmware installed, staged, recovery-capable, or active on a managed component.

The aim is a consistent way to associate SBOM artifacts with reported or measured system state, so fleet-scale composition and exposure can be assessed. It does not mean generating an SBOM from running software.

From association to fleet assessment



One way this could work, not a proposed architecture. The association layer is where the industry would need to agree.

Fleet principle: resolve once, reuse broadly, update intelligence independently, and never hide missing coverage.

Where could this information live?

- **Inventory: what is claimed**
  - SoftwareInventory and RelatedItem link firmware to devices and could support SBOM associations
- **Attestation: what is measured**
  - ComponentIntegrity and SPDM provide authenticated measurements that can help confirm deployed firmware matches a known artifact
  - Attestation neither generates an SBOM nor determines CVE applicability

Missing today: one interoperable way to tie artifact, SBOM, image state, provenance, and measurement together

Call to Action

- **Problem to solve:** There is no standard way to associate SBOMs with reported or measured fleet state, so exposure at fleet scale still rests on assumptions
- Agree on a minimum association record that any BMC and any management interface can carry
- Take it to the OCP Hardware Management project, working with the DMTF Redfish Forum
- Where do SoftwareInventory, RelatedItem, and ComponentIntegrity already cover this, and where do they stop?

Open Discussion

